The Journal of Open Source Software

kicad-pycbnew: Autonomous generation of KiCad PCBs in Python

Vincent Raspal [1¶]

1 Université Clermont Auvergne, CNRS/IN2P3, LPC, Clermont-Ferrand, 63178, France ¶ Corresponding author

DOI: 10.xxxxxx/draft

Software

- Review
- Repository
- Archive

Editor:

Submitted: 10 November 2025
Published: unpublished

License
Authors of papers retain copyright and release the work under a Creative Commons Attribution 4.0 International License (CC BY 4.0).

DRAFT

Summary

`kicad-pycbnew` is a Python module for generating electronic board designs from scratch in KiCad's native and open format (`.kicad_pcb`). Unlike other existing solutions, `kicad-pycbnew` does not require KiCad to be installed on the machine, nor an active session, to operate (however, it is not intended to replace KiCad for graphical rendering, zone filling, DRC checks, Gerber generation, or other advanced design and manufacturing features). `kicad-pycbnew` enables the creation of complex PCBs: it allows placement of component footprints, copper tracks, nets (logical connections between component pads to ensure proper electrical connections), vias, zones, and graphic elements (lines, rectangles, polygons, circles, arcs, text). All of this is possible without the usual prerequisite step of producing a symbolic electronic schematic.
In general, `kicad-pycbnew` proves especially powerful in scenarios where copper geometries are the core of the PCB design and components, while possibly numerous, play a secondary role or follow regular patterns.

Statement of need

In modern particle detectors, including Resistive Plate Chambers (RPCs), printed circuit boards (PCBs) are widely used for signal readout. RPC readout PCBs typically contain few electronic components, but their design demands extreme precision. The key challenge is the definition of the readout strips, which are generally a few centimeters wide and may vary in width along their length to adapt to non-rectangular shapes. That situation is relatively common as detectors and RPCs often have very constrained layouts.

Signal traces collect the induced charges from both ends of each strip and route them to connectors leading to the Front-End Board (FEB), which hosts ASICs and FPGAs. Impedance control (typically 50 Ω) is critical to maintain signal integrity and velocity over the meter-scale dimensions of the boards (1–2 meters). These aspects are central to several detector-electronics developments. This is the case for the CMS experiment at CERN (Meola, 2020; Shchablo et al., 2020), and it will also be true for the COMET CRV (Cosmic Ray Veto) at KEK (see Section 7 of the COMET Phase-I Technical Design Report (Comet Collaboration, 2020)).

Unlike conventional PCB design, which focuses on component placement and routing, RPC readout boards require careful optimization of copper geometry (strips, traces), and integration within the detector's mechanical constraints. Other signal-carrying boards in the readout chain — for example, those transmitting signals from the RPC readout boards to Front-End Boards, or from Front-End Boards to Back-End Boards — face similar challenges. Together, these constraints ensure uniform detector performance and minimal signal distortion. Such constraints are common across RPC-related R&D efforts (Bielewicz, 2023; Calice Collaboration, 2021). Designing these boards requires a high degree of automation, as each modification of

strip geometry, via placement, or zone shape would otherwise force the designer to start again from scratch, making iterative testing slow and tedious. Conventional EDA toolchains help with the logical connections between components, but provide little support for the detailed structuring of copper, including zone (representing readout strips or pads) shaping and precise track routing. `kicad-pycbnew` was developed to address this gap, offering a dedicated tool for automated and programmable copper layout tailored to RPC readout boards (and many more electrical applications in general).

To further support reproducibility and illustrate practical usage, two advanced example scripts are provided, `09_RPC_readout_board.py` and `10_RPC_extension_board.py`, following eight introductory ones that each demonstrate a specific capability of kicad-pycbnew.

The first script generates a trapezoidal RPC readout PCB, closely following the layout shown in Figure 4 of (Shchablo et al., 2020) (see Figure 1 below). The second produces an extension board designed to connect a Front-End Board to the readout board (see Figures 2 and 3 below). As the extension board also merges traces in pairs, it will be used by the CRV of the COMET experiment to save FEB channels during the low-intensity phase. Running these examples allows users and reviewers to reproduce the generation of fully formed boards, demonstrating how `kicad-pycbnew` handles complex strip geometries, vias, and zone placement in a real-world detector application.

Existing tools support PCB automation to some extent, but their capabilities fall short for the specific demands of large-scale detector readout board design. For example, the official KiCad Python API allows scripted panelization, manufacturing data export, and basic manipulation of existing `.kicad_pcb` files, but it requires a running KiCad instance and provides only limited low-level access to the design. While it allows placement of footprints, tracks, and vias via scripting, it remains a general-purpose interface without high-level constructs for features such as variable-width readout strips or impedance-controlled transitions.

In contrast, `kicad-pycbnew` was developed and applied by the author in the CRV implementation for the COMET experiment, providing high-level, programmable constructs to handle complex copper geometries and iterative design requirements that conventional EDA tools cannot easily manage. It offers concise primitives for generating parallel trace bundles from a model path, replacing corners with circular arcs (see Figure 4), creating via grids, producing multi-segment connections between arbitrary points, and many more, all of which are demonstrated in the accompanying `example scripts`.

Tool	Main Features	Limitations for RPC-style PCB design
KiCad Python API (official)	Scripted access to pcbnew through a running KiCad instance; placement of footprints, tracks, and vias (The KiCad Development Team, 2025).	Requires KiCad to be running; general-purpose API; lacks high-level constructs for complex geometries or automated generation of strips and impedance-controlled traces.
`kicad-skip`	Manipulate existing PCB and schematic files via S-expression parsing (Deegan, 2023).	Can only edit existing files; no support for creating boards from scratch or for advanced layout patterns like non-rectangular strips.
`KiCadFiles`	Python library for parsing and editing KiCad files, focused on file I/O and basic manipulation (Steffen, 2024).	File-centric; does not provide high-level functions for automated strip creation, impedance transitions, or mass parallel track generation.

Tool	Main Features	Limitations for RPC-style PCB design
`kicad-pcb-api`	File-based operations on .kicad_pcb files; advanced placement algorithms; Freerouting integration; AI-assisted placement (circuit-synth, 2023).	Cannot directly handle arbitrary strip geometries or impedance-controlled track layouts for RPC boards; mainly focuses on component placement and routing automation.
`kicad-parts-placer`	Automatic placement of PCB parts based on predefined rules and libraries (Hobbs, 2023).	Limited to component placement; no tools for strip shaping, vias zoning, or precise impedance control.
`kicad-pycbnew`	Full-featured for detector PCBs: programmable placement of footprints, nets, tracks, vias; arbitrary-shape zones; automated impedance transitions; generation of equidistant parallel tracks; creation of via zones. (Raspal, 2025)	Fully addresses specialized RPC PCB needs more efficiently than generic tools.

Illustrations

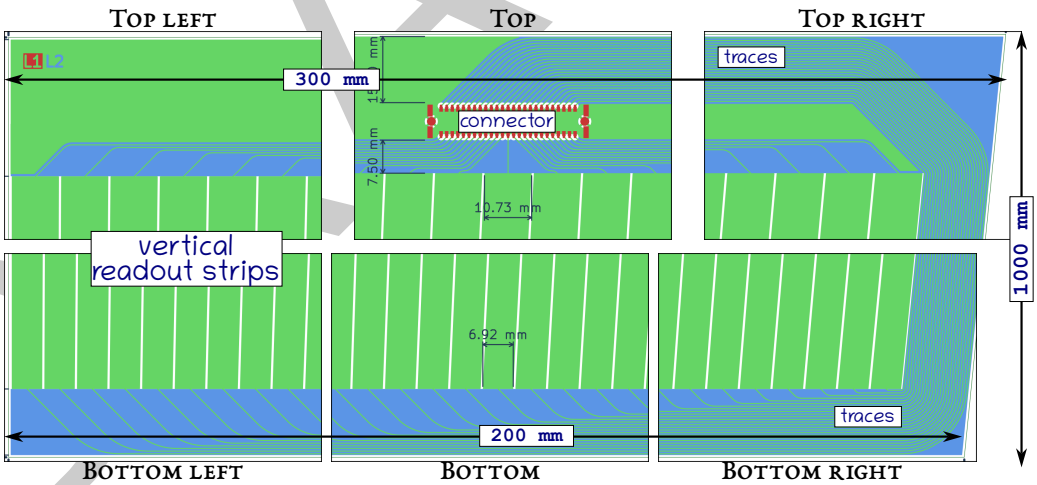

Figure 1: Sections of a 1000 × 300 mm² RPC readout board generated with the script `09_RPC_readout_board.py`. The figure shows the strip layout, the returning parallel traces and their connections to footprint pads, the via transitions, and the inclusion of user-defined annotations such as dimensions. A complete view of the board is available in the project's GitLab repository.

DRAFT

The Journal of Open Source Software

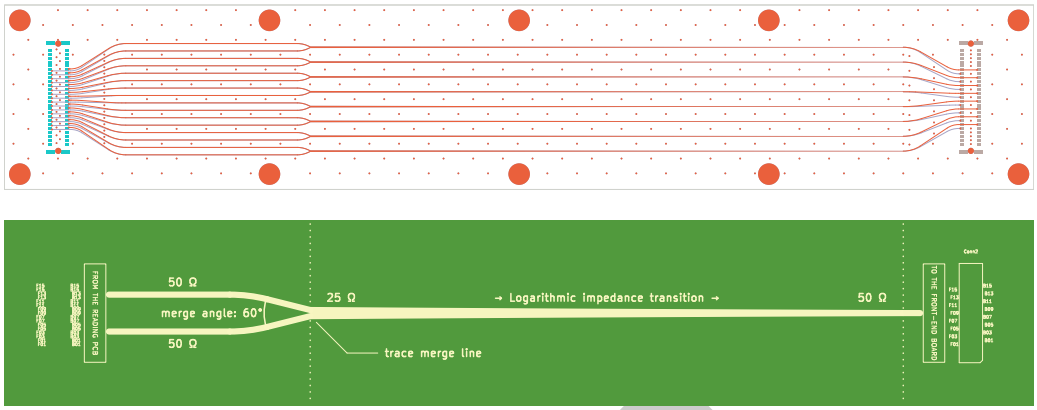

Figure 2: Inner copper and top silkscreen layers of an extension board designed with `kicad-pycbnew` to be placed between an iRPC readout board and a Front-End Board (FEB), allowing the FEB to be positioned farther away to reduce noise and radiation exposure. The script `10_RPC_extension_board.py` is provided as a usage example. It shows traces emerging from a connector, diverging, and then merging in pairs, finally passing through a logarithmic impedance transition between traces of different widths. The traces are routed on a structured grid with multiple ground vias, including user-defined exclusion zones (`kicad-pycbnew` feature) to maintain signal integrity.

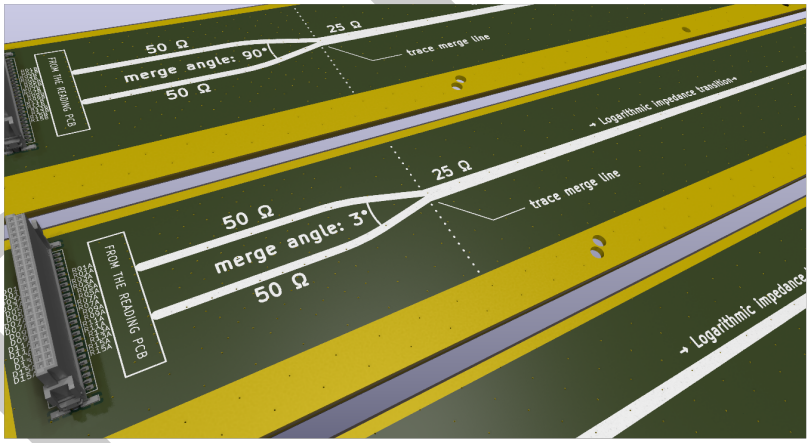

Figure 3: 3D view rendered in KiCad of a set of three extension boards designed through `kicad-pycbnew` and the `10_RPC_extension_board.py` script.

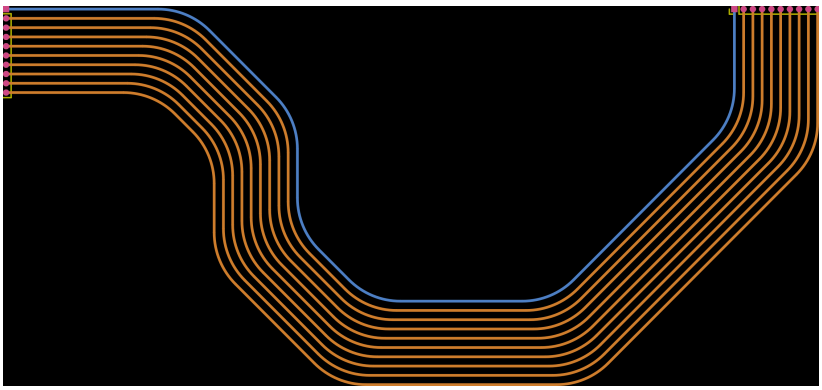

Figure 4: Example of a `kicad-pycbnew` feature: automated creation of a set of curved, parallel traces following a reference path (here in purple).

DRAFT

Acknowledgements

This work was supported by the COMET collaboration and by Université Clermont Auvergne.

References

Bielewicz, M. et al. (2023). Practical implementation of an analogue and digital electronics system for a modular cosmic ray detector—MCORD. *Electronics*, *12*(6), 1492. https://www.mdpi.com/2079-9292/12/6/1492

Calice Collaboration. (2021). Development of novel designs of resistive plate chambers. *Instruments*. https://cds.cern.ch/record/2861342/files/document.pdf

circuit-synth. (2023). *Kicad-pcb-api*. https://pypi.org/project/kicad-pcb-api/.

Comet Collaboration. (2020). COMET phase-I technical design report. *Progress of Theoretical and Experimental Physics*, *2020*(3), 033C01.

Deegan, P. (2023). *Kicad-skip*. https://github.com/psychogenic/kicad-skip.

Hobbs, S. (2023). *Kicad-parts-placer*. https://pypi.org/project/kicad-parts-placer/.

Meola, S. et al. (2020). Towards a two-dimensional readout of the improved CMS resistive plate chamber. *arXiv Preprint arXiv:2006.00576*.

Raspal, V. (2025). *Pycbnew*. https://gitlab.com/vinraspa/kicad-pycbnew.

Shchablo, K., Laktineh, I., Gouzevitch, M., Combaret, C., & Mirabito, L. (2020). Performance of the CMS RPC upgrade using 2D fast timing readout system. *Nuclear Instruments and Methods in Physics Research Section A: Accelerators, Spectrometers, Detectors and Associated Equipment*, *958*, 162139. https://doi.org/10.1016/j.nima.2019.04.093

Steffen, W. (2024). *KiCadFiles*. https://steffen-w.github.io/KiCadFiles/.

The KiCad Development Team. (2025). *KiCad python API*. https://gitlab.com/kicad/code/kicad-python/.

DRAFT